

Forecasting Coffee Shop Ingredient Demand from Sales and Recipe Data

Nazarii Ivakhniuk
Department of Informatics

Universität Hamburg

Hamburg, Germany

nazarii.ivakhniuk@studium.uni-hamburg.de

Gustav Arnold
Department of Informatics

Universität Hamburg

Hamburg, Germany

gustav.arnold@studium.uni-hamburg.de

Abstract—This demo paper presents a database-backed workflow for forecasting ingredient demand in a coffee shop. The system combines coffee sales stored in SQLite with recipe data stored in MongoDB to derive daily ingredient usage for items such as milk and sugar. The resulting time series is forecast with additive Holt-Winters exponential smoothing, using weekly seasonality to model recurring demand patterns. The demo shows how product-level sales records can be transformed into ingredient-level planning information for purchasing and stock management.

Index Terms—ingredient demand forecasting, time series forecasting, Holt-Winters exponential smoothing, SQLite, MongoDB, coffee sales, inventory planning

I. INTRODUCTION

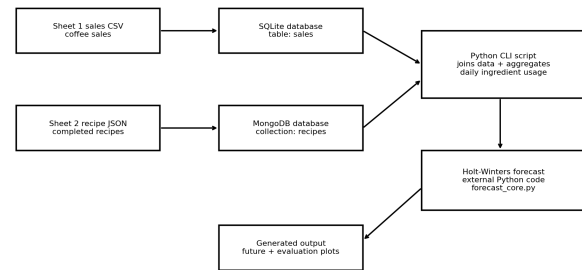
Coffee shops have to plan ingredient purchases before the actual demand is known. If too little milk, sugar, or other ingredients are available, products may become unavailable during the day. If too much is ordered, the shop increases storage costs and risks wasting ingredients. Sales records alone only show which coffee products were sold, but they do not directly answer how much of each ingredient was consumed. Therefore, forecasting ingredients directly is more helpful for planning purchases than only looking at sold coffee products.

This demo addresses the problem by combining coffee sales data with recipe data. The sales data is stored in SQLite, while the recipe data is stored in MongoDB. A Python script reads both sources, maps each sold coffee to the ingredients required by its recipe, and aggregates the result into a daily ingredient usage time series. The resulting time series is then forecast with additive Holt-Winters exponential smoothing, using a weekly season length to model recurring weekday patterns. The demo focuses on milk and sugar, but the same workflow can be applied to any ingredient contained in the recipe database.

II. ARCHITECTURE OVERVIEW

The system is organized as a small data pipeline, shown in Fig. 1. The coffee sales CSV from the first exercise sheet is imported into a SQLite database table called `sales`. The completed recipe JSON from the second exercise sheet is imported into a MongoDB collection called `recipes`. This also fits the structure of the data: the sales data is tabular, while the recipes are easier to store as documents.

Architecture for Exercise Sheet 3 Forecasting



Data is first stored in databases. Forecasting runs in an external Python script after aggregation.

Fig. 1. Architecture of the demo pipeline from source data to ingredient forecasts.

The main Python script reads both databases and connects each sold coffee product with its recipe. For every sale, the script looks up the amount of the selected ingredient required by the corresponding recipe. It then sums these values per day and fills missing dates with zero usage, producing a complete daily time series for the chosen ingredient. The implementation also normalizes some recipe names for planning purposes: for example, Milk foam and Milk or water are counted as milk.

A. Forecasting Method

Forecasting is implemented in a separate Python module using additive Holt-Winters exponential smoothing, also known as triple exponential smoothing [1], [2]. The method decomposes the ingredient usage series into three parts: a level component for the current baseline demand, a trend component for longer-term increases or decreases, and a seasonal component for recurring patterns. This is suitable for the demo because coffee-shop demand can vary strongly between weekdays while still following a repeated weekly rhythm.

The model receives the daily ingredient usage values ordered by date. For the first forecast run, the most recent known days are held back as test data. The model is trained on the earlier part of the series and predicts the held-back period, which allows the demo to compute the mean absolute

error between actual and predicted usage. For the second run, the model is trained on the complete daily series and predicts the next days of future ingredient demand. The default configuration uses a season length of seven days, a 14-day forecast horizon, and smoothing parameters $\alpha = 0.05$, $\beta = 0.01$, and $\gamma = 0.40$. These parameters can be changed from the command line to demonstrate how the forecast reacts to different assumptions about level, trend, and seasonality.

III. DEMONSTRATION WORKFLOW

In the demo, we show the entire process from loading the data to generating the forecast plots. During the demo, the user selects an ingredient, runs the forecasting script, and inspects the generated evaluation and future forecast outputs. The main scenario is demand planning for milk, with sugar as a second example that shows the same workflow for a different ingredient.

A. Setup

The demo runs locally on Ubuntu Linux with Python 3.12.3. SQLite 3.45.1 is used for the sales table, and MongoDB is used for the recipe collection at `mongodb://localhost:27017/`. The Python implementation uses `pandas` for data processing, `pymongo` for MongoDB access, and `matplotlib` for the plots.

The forecasting script is controlled through command-line parameters. The flag `--ingredient` selects the ingredient to forecast, for example Milk or Sugar. The smoothing parameters `--alpha`, `--beta`, and `--gamma` control how strongly the model updates the level, trend, and seasonal components. The flag `--season-length` defines the seasonal cycle, which is set to seven for weekly patterns in daily data. The flags `--horizon` and `--test-days` define the number of future days to forecast and the number of known recent days used for evaluation. Finally, `--no-setup` skips reloading the SQLite and MongoDB databases when the data has already been imported.

B. User Experience

At the beginning of the demo, the setup script loads the sales CSV into SQLite and the recipe JSON into MongoDB. The user then runs the command-line forecasting script, for example with the default ingredient Milk or with `--ingredient Sugar`. The script prints the selected ingredient, the Holt-Winters parameters, and the evaluation error. It also creates three CSV files: the aggregated daily usage, the evaluation forecast, and the future forecast.

The main visual result is the future forecast plot shown in Fig. 2. The blue line shows the recent historical milk usage, while the orange points show the predicted demand for the 14 days following the available sales data. The forecast continues the weekly structure learned by the Holt-Winters model, but it should be interpreted as an operational estimate rather than an exact prediction because the historical sales data is volatile. In the demo, this plot is used to discuss how much milk the shop should expect to need in the near future and how parameter

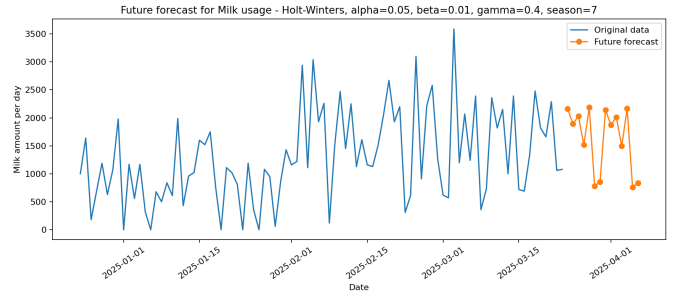


Fig. 2. Future milk usage forecast generated by the demo for the 14 days following the available sales data.

changes affect the forecast shape. A second generated plot compares predictions with known recent data and is used for evaluation, but it is not included here to keep the paper focused.

C. Conclusion

The demo shows how operational sales data can be transformed into ingredient-level planning information. The main point of the demo is that it connects the database part of the exercise with a practical forecasting task.

REFERENCES

- [1] C. C. Holt, "Forecasting seasonals and trends by exponentially weighted moving averages," Carnegie Institute of Technology, Office of Naval Research Memorandum, 1957.
- [2] P. R. Winters, "Forecasting sales by exponentially weighted moving averages," *Management Science*, vol. 6, no. 3, pp. 324–342, 1960.